Vectoria: Supercomputer-driven vector database for private LLM retrieval

User Manual, version 2.0

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Germany, Bulgaria, Austria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, Greece, Hungary, Ireland, Italy, Lithuania, Latvia, Poland, Portugal, Romania, Slovenia, Spain, Sweden, France, Netherlands, Belgium, Luxembourg, Slovakia, Norway, Türkiye, Republic of North Macedonia, Iceland, Montenegro, Serbia under grant agreement No 101101903.

This page intentionally blank

Summary

Acronyms and Terminology 3
1. Introduction 5
2. EuroCC: Objectives 5
3. Summary 5
4. Service Description 6
5. System Components 6
6. Self-Deployment Guide 7
Prerequisites 8
Get the Source Code 8
Install Locally with uv: minimal profile 8
Install locally with uv: optional features 9
Install Locally with Docker 10
Leonardo / CINECA Deployment 11
Reconnect and stop the service 13
7. Using Vectoria 13
8. Troubleshooting 15
9. Appendix A: Technical architecture 15
Indexing and answering workflows 16
10. Appendix B: Advanced Configuration 18
Environment variables and model providers 19
General settings and model files 19
Extraction and chunking 20
Vector storage and embeddings 21
Retrieval and reranking 21
Wiki 22
Entity graph 22
Conversation history and guardrails 23
Inference settings and local model servers 24
MLflow observability 24
Applying configuration changes 25
11. Appendix C: API and command line use 25
Authentication and collection selection 25

Upload through the API 26
Command-line server and terminal chat 26
12. Application information and contributions 28

Acronyms and Terminology

AI	Artificial Intelligence	Field of study in computer science that develops and studies intelligent machines.
API	Application Programming Interface	A programmatic interface used by clients to interact with Vectoria.
BM25	Best Matching 25	A lexical ranking method used in Vectoria's hybrid retrieval.
CLI	Command-Line Interface	A text-based interface used to interact with software or operating systems.
GPU	Graphics Processing Unit	Specialized hardware designed for accelerating computational tasks, especially in AI and ML.
HPC	High-Performance Computing	Computing systems with high-speed processing capabilities used for complex computations.
LLM	Large Language Model	A type of AI model trained on vast amounts of text to perform natural language understanding and generation.
MMR	Maximal Marginal Relevance	A ranking technique that optimizes information relevance and diversity in retrieved results.
NER	Named Entity Recognition	Identification of entities used to construct the knowledge graph.
PMI	Pointwise Mutual Information	A statistical measure used to filter noisy graph relationships.
RAG	Retrieval-Augmented Generation	A technique that combines information retrieval and generative AI for producing contextually relevant outputs.
SLURM	Simple Linux Utility for Resource Management	The workload scheduler used on Leonardo.
Tenant	Isolated Vectoria workspace	A logical customer or organization selected with X-Tenant-ID.

1. Introduction

Vectoria is an on-premise Retrieval-Augmented Generation (RAG) service for asking questions over private organizational documents. It was built for the IT4LIA project and is designed to keep document processing, indexes, conversation history and model inference under the operator's control. The software can run directly with uv, as a Docker Compose service, or on CINECA's Leonardo supercomputer through an automated Ansible deployment.

This manual is intended for system administrators, technical operators and users who need to install, configure, run and use Vectoria. It describes the currently supported workflows and should be read together with the repository README files for deployment-specific troubleshooting and parameter details.

Privacy depends on the selected inference endpoint. When Vectoria is connected to a model hosted locally or on the organization's HPC infrastructure, documents and retrieved context remain on premises. Connecting an external OpenAI-compatible provider can transmit prompts and retrieved context to that provider and must therefore be evaluated against the organization's data-handling policy.

2. EuroCC and IT4LIA AI Factory: Objectives

EuroCC 2 is a project which works to identify and address the skills gaps in the European High-Performance Computing (HPC) ecosystem and coordinate cooperation across Europe to ensure a consistent skills base. The mission of EuroCC is to improve the technological readiness of Europe. In particular, the role of EuroCC2 is to establish and run a network of more than 30 NCCs across the EuroHPC Participating States. The NCCs act as single points of access in each country between stakeholders and national and EuroHPC systems. They operate on a regional and national level to liaise with local communities, in particular SMEs, map HPC competencies and facilitate access to European HPC resources for users from the private and public sector.



Figure 1: NCCs across Europe

EuroCC2 delivers training, interacts with industry, develops competence mapping and communication materials and activities, and supports the adoption of HPC services in other related fields, such as quantum computing, artificial intelligence (AI), high performance data analytics (HPDA) to expand the HPC user base.

IT4LIA AI Factory is a strategic European initiative, co-funded by the EuroHPC Joint Undertaking, aimed at accelerating the development and adoption of Artificial Intelligence (AI) in Italy and across Europe. Hosted and

operated by CINECA, with the participation of Austria and Slovenia, IT4LIA is part of the European network of AI Factories and seeks to strengthen Europe's technological competitiveness, innovation capacity, and digital sovereignty.

IT4LIA leverages the EuroHPC Leonardo supercomputer and the advanced computing infrastructure located at the Bologna Tecnopolo, including the LISA system, GAIA cloud infrastructure, and a forthcoming AI-optimised supercomputer. These resources provide a computing continuum supporting the entire AI lifecycle, from data preparation and processing to model development, training, fine-tuning, testing, and inference.

The AI Factory acts as a single point of access to advanced AI infrastructure, expertise, and services for startups, Small and Medium-sized Enterprises (SMEs), research institutions, public administrations, and industrial stakeholders. Particular attention is given to lowering the barriers to AI adoption by providing access to computing resources, technical assistance, training, and tailored support for developing and scaling AI solutions.

3. Summary

Vectoria combines document ingestion, hybrid retrieval and an agentic question-answering interface in a multi-tenant service. Each answer is generated from retrieved evidence and cites its source files. The current application provides the following capabilities:

- Hybrid search that fuses BM25 lexical matches with semantic vector retrieval, followed by reranking and optional MMR diversity.
- PDF and DOCX ingestion with native extraction or Docling/OCR, recursive or semantic chunking, and persistent Chroma, FAISS or Qdrant indexes.
- A per-tenant Markdown wiki that synthesizes concepts from uploaded documents and a per-tenant entity graph for relational queries.
- A multi-turn ReAct agent with SQLite-backed history, tool-call limits and explicit human approval before document deletion.
- An OpenAI-compatible API for integration with OpenWebUI and other clients, plus a browser upload page and command-line workflows.
- Isolated tenant data selected with X-Tenant-ID; OpenAI-compatible model identifiers use vectoria-<tenant_id>.
- Integration with tool-capable OpenAI-compatible inference endpoints, including appropriately configured vLLM, Ollama and llama-server services. The selected endpoint and model must support the tool calls required by Vectoria’s agent. Direct Hugging Face and llama.cpp bindings are also implemented, but their compatibility with the complete interactive agent workflow must be verified separately.
- Local CPU-oriented defaults and an automated, GPU-optimized Leonardo deployment with vLLM, OpenWebUI, SLURM and SSH tunnels.
- Optional MLflow observability for local Docker deployments.

4. Service Description

Vectoria helps you find information in your organisation’s documents through a browser chat. Upload the files once, wait for them to become searchable, and ask questions in everyday language. Answers can identify the documents used so that you can check the original information.

The service can run on a local computer with Docker or on CINECA Leonardo. A person administering the service installs it and connects a language model. Other users need only the browser addresses and the name of their document collection.

- Upload documents into the collection assigned to you.
- Ask questions, request summaries and compare information across files.
- Check the cited sources before relying on an answer.

5. System Components

- **Configuration:** YAML configuration controls extraction, chunking, vector storage, retrieval, reranking, wiki, graph, history, guardrails and inference. Docker uses etc/docker/config.yaml; HPC renders its configuration from ansible/roles/vectoria/templates/default_config.j2.
- **Document preprocessing:** Native extraction handles ordinary PDF and DOCX content; Docling can be enabled for layout-aware extraction and OCR. Text is cleaned and divided with recursive or semantic chunking while source metadata is retained.
- **Indexes:** A tenant receives a semantic vector store and, when hybrid retrieval is enabled, a persisted BM25 index. Chroma, FAISS and Qdrant are supported.
- **Retrieval and reranking:** Lexical and semantic candidates are fused, reranked and optionally filtered with minimum-per-source and MMR diversity rules.
- **Knowledge layers:** The wiki stores synthesized Markdown pages. The graph uses GLiNER entity extraction, configurable PMI filtering, entity resolution and co-occurrence, spaCy or LLM relationships.
- **Agent and tools:** A LangGraph ReAct agent manages the conversation and decides when to retrieve, inspect the wiki or graph, list documents, or request deletion. SQLite checkpoints maintain continuity between turns.
- **API and user interfaces:** FastAPI exposes document operations and OpenAI-compatible /v1 endpoints. OpenWebUI provides chat, and /upload provides a browser-based upload interface.
- **Inference backend:** The application can call any compatible remote or on-premise endpoint, or use supported local engines. The endpoint, key, model name and timeout are environment-driven in Docker.

6. Self-Deployment Guide

Vectoria can run directly on a host with uv, in Docker Compose, or on Leonardo through Ansible.

Deployment	What it starts	Language model
Local with uv	Vectoria API and upload page	Connect an existing inference endpoint
Local with Docker Compose	Vectoria API, upload page and OpenWebUI	Connect an existing inference endpoint, or configure the optional llama-server integration
Leonardo with Ansible	Vectoria, vLLM and OpenWebUI inside a scheduled job	Deployment downloads and serves the configured model

The local uv installation does not start OpenWebUI. Use the terminal chat client, an API client, or a separately configured chat interface.

The configuration files supplied with the repository have different purposes:

File	Purpose
`.env.example`	Environment-variable template; copy it to .env
`etc/local/config.yaml`	Minimal configuration for direct host execution
`etc/docker/config.yaml`	Configuration packaged into the Docker image
`ansible/roles/vectoria/vars/config.yml`	HPC deployment settings
`ansible/roles/vectoria/templates/default_config.j2`	Template used to generate the HPC application configuration

The HPC files etc/hpc/default_config.yaml and etc/hpc/vectoria_hpc_launcher.job are generated on Leonardo during deployment. Their absence from the initial checkout is expected.

Prerequisites

For local host execution, use Python 3.12 or 3.13, uv, Git and a shell. Run the commands below from the repository root.

For Docker, install Docker Engine or Docker Desktop with the Compose plugin. Python and uv are installed inside the backend image.

Initial installation and model downloads require internet access. Embedding models, reranking models and optional OCR or graph models have their own download and memory requirements, separate from the language model.

Local installations require an inference endpoint and model that support tool calls. Obtain:

- The endpoint's OpenAI-compatible base URL, normally including /v1.
- Its authentication key, or a nonempty dummy value if authentication is disabled.
- The exact served model name.

The supplied Docker backend has a 6 GB memory limit. This is a container setting, not a guarantee that every document or optional feature will fit. Adjust resources for the selected workload.

Get the Source Code

```
git clone https://github.com/Eurocc-Italy/Vectoria.git
cd Vectoria
```

Create the environment file:

```
cp .env.example .env
```

Edit .env before starting the application. It contains placeholders, not usable credentials.

The populated .env is excluded from Git and Docker builds. Do not place credentials directly in tracked YAML configuration files.

Install Locally with uv: minimal profile

The minimal profile uses native extraction, CPU embeddings and local Qdrant storage. Wiki synthesis, graph extraction and reranking are disabled. It does not provide OCR for scanned documents.

Install the API dependencies:

```
uv sync --extra api --no-dev
```

Set these variables in .env:

```
LLM_PROVIDER=custom
LLM_BASE_URL=https://your-model-server.example/v1
LLM_API_KEY=replace-with-your-model-key
LLM_MODEL=replace-with-the-served-model-name
LLM_TIMEOUT=120
PERMITTED_KEYS=
```

LLM_BASE_URL points to the upstream language model service. It must not point back to Vectoria's own /v1 endpoint.

Create the runtime directories

```
mkdir -p data/index data/uploads data/history data/logs
```

Start Vectoria:

```
uv run python bin/vectoria --config etc/local/config.yaml serve \
  --index-dir data/index \
  --upload-dir data/uploads \
  --host 127.0.0.1 \
  --port 8000
```

Open:

- Upload page: http://localhost:8000/upload
- Health endpoint: http://localhost:8000/health
- OpenAI-compatible API base: http://localhost:8000/v1

The application reads the repository-root .env automatically. Existing exported environment variables take precedence. Sourcing .env is therefore optional for host execution.

Always pass --config explicitly. The CLI's current implicit configuration path refers to a file not supplied in this checkout.

Upload one short document and wait for indexing to complete. In another terminal, start terminal chat:

```
uv run python bin/chat --tenant default
```

This terminal client assumes Vectoria token authentication is disabled.

Install locally with uv: optional features

To install API, Docling, graph and observability dependencies together:

```
uv sync --extra api --extra docling --extra graph \
  --extra observability --no-dev
```

If graph relation labeling uses the supplied Italian spaCy model, download it separately:

```
uv run python -m spacy download it_core_news_sm
```

Installing packages does not enable the corresponding application features. Create a working configuration:

```
cp etc/local/config.yaml etc/local/config.custom.yaml
```

Edit the existing settings in that file:

Feature	Configuration change
PDF OCR fallback	Set data_ingestion.extraction.backend to auto
Docling extraction	Set data_ingestion.extraction.backend to docling
Cross-encoder reranking	Set reranker.enabled to true
Entity graph	Set graph_store.enabled to true
Generated wiki	Set wiki.enabled to true
Query rewriting	Set retriever.multi_query to true

Docling may require additional native libraries and OCR resources on the host. The Dockerfile includes system packages for its Docker environment; installing the Python extra alone does not reproduce those system packages on every operating system.

Wiki synthesis and query rewriting currently send the backend-specific request option chat_template_kwargs.enable_thinking=false. Enable them only after confirming that the inference endpoint accepts this option. An endpoint that rejects it requires an application compatibility change or these features must remain disabled.

Start the enhanced configuration with:

```
uv run python bin/vectoria --config etc/local/config.custom.yaml serve \
  --index-dir data/index \
  --upload-dir data/uploads \
  --host 127.0.0.1 \
  --port 8000
```

MLflow requires a tracking service and a tracking URI in addition to the observability package. See Appendix B.

Install Locally with Docker

Docker Compose starts Vectoria and OpenWebUI. It does not start a general external language model service.

In the repository-root .env, set:

```
DATA_DIR=./data
CONFIG_PATH=etc/docker/config.yaml
LLM_PROVIDER=custom
LLM_BASE_URL=https://your-model-server.example/v1
LLM_API_KEY=replace-with-your-model-key
LLM_MODEL=replace-with-the-served-model-name
LLM_TIMEOUT=120
PERMITTED_KEYS=
```

For an inference service running on the Docker host, use a host-reachable URL. For example:

```
LLM_BASE_URL=http://host.docker.internal:11434/v1
```

Inside the Vectoria container, localhost refers to that container.

The supplied Docker configuration enables additional features and includes vLLM-style thinking parameters. Before using a different endpoint:

- Remove the inference_engine.model_kwargs block if the endpoint does not accept those additional parameters.
- Disable wiki.enabled and keep retriever.multi_query disabled if the endpoint rejects the auxiliary thinking parameter described above.
- Verify that the selected model supports tool calls.

To customize the Docker configuration and persist conversation history, create a working copy:

```
cp etc/docker/config.yaml etc/local/config.custom.yaml
mkdir -p data/history
```

If DATA_DIR differs from ./data, create history under that directory instead.

In the copied YAML file, change the existing database setting to:

```
chat_history:
  # Retain the other existing chat_history settings.
  checkpoint_db_path: /data/history/chat_history.db
```

Create compose.override.yaml in the repository root:

```
services:
  vectoria-api:
    volumes:
      - ./etc/local/config.custom.yaml:/app/etc/docker/config.yaml:ro
      - ${DATA_DIR:-./data}/history:/data/history
```

Keep CONFIG_PATH=etc/docker/config.yaml in .env for this mount arrangement. Compose automatically reads compose.override.yaml.

```
Check and start the deployment:
docker compose config --quiet
docker compose up --build -d
docker compose ps
```

Open:

- Upload page: http://localhost:8000/upload
- Health endpoint: http://localhost:8000/health
- Chat interface: http://localhost:8080
- OpenAI-compatible API base: http://localhost:8000/v1

An HTTP response from /health confirms that the API process is reachable. Verify the complete installation by uploading a short document, waiting for indexing, selecting the matching collection in OpenWebUI and obtaining an answer grounded in that document.

Inspect backend logs when necessary:

```
docker compose logs --tail=100 vectoria-api
```

PERMITTED_KEYS controls access to Vectoria, whereas LLM_API_KEY authenticates Vectoria to the upstream model. The bundled upload page does not send Vectoria's required authentication header. See Appendix C before enabling token authentication.

Leonardo / CINECA Deployment

Run Ansible from your own Linux computer. You need Git, uv, an SSH client, rsync, a CINECA account, a valid allocation and a usable SSH certificate.

Create a separate local environment for Ansible:

```
uv venv --python 3.12 .venv-ansible
source .venv-ansible/bin/activate
uv pip install -r ansible/requirements.txt
```

Add the following variables to the repository-root .env:

```
CINECA_USER=your-username
CINECA_SSH_KEY=/absolute/path/to/.ssh/cineca_key
ANSIBLE_HPC_ACCOUNT=your-budget-account
VECTORIA_API_TOKEN=replace-with-a-long-random-token
```

The current automatic tunnel task uses ~/.ssh/cineca_key, even when another CINECA_SSH_KEY is configured. Use that identity for the automatic workflow. Using a different identity requires correcting the tunnel task or opening SSH forwarding manually.

CINECA_EMAIL is additionally required if using the optional etc/hpc/leonardo_connect.sh helper. VECTORIA_REPO_URL is not required by the current deployment, which synchronizes this checkout.

Review ansible/inventory/hosts and ansible/roles/vectoria/vars/config.yml before deployment. In particular, check:

Settings	Purpose
account, partition, qos, job_time	Allocation and scheduler settings
nodes, gpus, cpus	Requested resources
vllm_gpus, vectoria_gpus, vllm_tensor_parallel_single	Single-node GPU allocation
llm_model, vllm_image_url, vllm_tool_call_parser	Compatible model, server image and tool parser
vllm_gpu_memory_utilization	vLLM memory budget
Embedding, reranker and graph model settings	Models downloaded before execution
Local and remote port settings	Service and SSH-forwarding addresses

The supplied configuration requests one node with four GPUs and a 30-minute job. In single-node mode, it assigns separate GPU groups to vLLM and Vectoria. Multi-node mode follows a different Ray and parallelism path; do not assume the single-node GPU allocation applies unchanged.

Restricted models require approved access and authentication on Leonardo before the download tasks run.

Load the environment and deploy:

```
set -a
source .env
set +a

cd ansible
ansible-playbook -i inventory/hosts deploy.yml
```

The playbook synchronizes the source, prepares dependencies and images, downloads models, generates the HPC configuration and launcher, submits the scheduled job and opens SSH tunnels.

Default local addresses are:

- Upload page: http://localhost:8081/upload
- Chat interface: http://localhost:8082

The current automatic tunnels bind the forwarded ports to 0.0.0.0, making them reachable through the local machine's network interfaces where network policy permits. For a personal workstation deployment, use localhost-only forwarding if access from other machines is not intended.

Reconnect and stop the service

If you lose the Leonardo connection, run

```
ansible-playbook -i inventory connect.yml
```

from ansible/ with the environment loaded. Add -e job_id=1234567 to select a running job. On Leonardo,

```
squeue -u YOUR_USERNAME
```

shows your jobs and scancel JOB_ID stops the selected job.

For Docker,

```
docker compose stop
```

stops the services and

```
docker compose start
```

starts them again. Closing a browser does not stop Docker services or cancel a Leonardo job

7. Using Vectoria

Prepare your documents

The upload page accepts PDF, DOCX, XLSX, PPTX, HTML, Markdown, TXT and CSV. Start with one short, readable document. Scanned PDFs need text recognition, called OCR; this depends on the installation. Keep a copy of the original files outside Vectoria.

Upload no more than 20 files at a time, with each file no larger than 200 MB. Use distinct, descriptive filenames, such as leave_policy_2026.pdf. Upload only material approved for this service.

Upload and index

1. Open the upload page. In Tenant ID, enter the collection name assigned by your administrator. Leave default only if that is the intended collection. Names use lowercase letters, numbers, hyphens or underscores, up to 64 characters.
2. Drag files into the upload area or click sfoglia to select them. Review the file list.
3. Click “Carica e Indicizza”. Wait for the indexing completion message before asking questions. Upload completion alone does not mean the documents are searchable.
4. Open the chat page and choose the matching collection. For Tenant ID company-a, choose vectoria-company-a. Refresh the page or model list if it has not appeared.

The chat for the collection can be unavailable during indexing. Large files, scanned pages and the first model load take longer. Do not start another upload for the same collection while indexing is running.

Cancel or update documents

To stop indexing, click Annulla indicizzazione and wait before retrying. Processing already running can take a short time to end. Check the available documents after cancellation.

To remove a document, first ask the chat to list the available files. Then ask to delete the exact filename from the selected collection. With the supplied settings, Vectoria asks for confirmation. Check the file and collection before confirming.

To replace a file, keep a copy of the original, delete the obsolete version, wait for the collection to be ready and upload the replacement. Do not assume that uploading the same filename replaces every previously indexed passage.

Ask questions in the chat

1. Open the chat interface (OpenWebUI).
2. Select the vectoria- entry matching your collection. A generic vectoria entry may appear before any collection is ready.
3. Start a new chat and ask a question whose answer should be in the uploaded files. Name the document or topic.
4. Read the answer and use the source filename to check the original document. Continue with a follow-up question in the same chat.

For a first test, try something like: “Using staff_handbook.pdf, explain how to request annual leave and identify the source.” Replace the filename and topic with a document you actually uploaded.

Specific questions are easier to answer. For example, ask “Which travel expenses require approval?” rather than “Tell me about travel”. You can request a numbered procedure, a summary or a comparison. Ask the system to say when the documents do not contain enough information.

Check sources and answer quality

Vectoria is configured to identify sources, often with a label such as [Source: filename.pdf]. The reference may not be a clickable link. Check the relevant passage in your original document, particularly for important decisions.

If an answer is incomplete, name the file, narrow the question or split it into smaller questions. Check that the current document version is in the correct collection. Start a new chat when you change collection or topic; restate the subject if a long conversation loses context.

Some installations also provide a generated wiki and a graph of relationships between entities. You can ask for a topic summary or how two entities are connected. These optional features depend on the configuration and may require extra processing. Check their answers against the original documents too.

Data and privacy

The administrator chooses the language model. With an external endpoint, the questions and document passages needed to answer them are sent to that service. Confirm that the setup is suitable before uploading confidential material. A Tenant ID groups documents; it is not a personal login or access permission.

Docker stores uploads and indexes under the configured DATA_DIR, which defaults to ./data. OpenWebUI uses ./data/openwebui by default unless OPENWEBUI_DATA_DIR is changed.

The unmodified Compose file does not persist the configured conversation database. The history-directory mount described in Section 6 provides that persistence. Vectoria creates tenant-specific database filenames; back up the complete history directory rather than assuming there is only one chat_history.db.

Stop the services before taking a filesystem backup. Include uploads, indexes, history, OpenWebUI data, custom configuration and the protected .env file. Remote Qdrant storage and MLflow use separate persistence locations and require their own backups.

8. Troubleshooting

- **The page will not open:** Check the address. For Docker, run docker compose ps and inspect the backend log. For Leonardo, check that the job is RUNNING and reopen the tunnels with connect.yml.
- **The collection is missing:** Wait for indexing to finish, check Tenant ID and refresh the model list. Select the matching vectoria-collection entry. A generic vectoria entry does not mean documents are ready.
- **The upload is still processing:** Read the progress message and wait. Large scans and first-time model loading can be slow. Avoid another upload for the same collection.
- **Upload failed with 409:** Another indexing operation or a cancellation is still finishing. Wait before retrying.
- **Upload failed with 413 or 415:** Use at most 20 files per upload, each no larger than 200 MB, and a format accepted by the upload page.
- **Unauthorized or 401:** Ask the administrator to check authentication. The supplied upload page does not send an API token; authenticated clients need an x-token header. See Appendix C.
- **Too many requests or 429:** Wait before retrying. The service limits repeated chat and upload requests.
- **The answer is empty or the model fails:**
- Check the upstream URL, served model name, credentials, timeout and tool-call support. Check whether the endpoint accepts the configured chat_template_kwargs extension. Wiki synthesis and query rewriting also use that extension. Reasoning models may require a larger output-token budget.
- **A scanned document cannot be read:** Try a PDF with selectable text. Ask the administrator to check OCR dependencies and extraction settings.

When requesting help, include the time, deployment method, action, Tenant ID, software revision and visible error. For uploads, include the file type and size. Remove credentials and sensitive document content from logs and screenshots. Report software problems at https://github.com/IT4LIA-AIFactoryEU/vectoria; IT4LIA access support is available at support-it4lia@cineca.it. For service-access questions, contact the administrator of your installation.

9. Appendix A: Technical architecture

This detailed diagram is a reference for the internal pipeline. It is not needed for the browser tasks described in Usage. The active configuration determines which optional components are enabled.

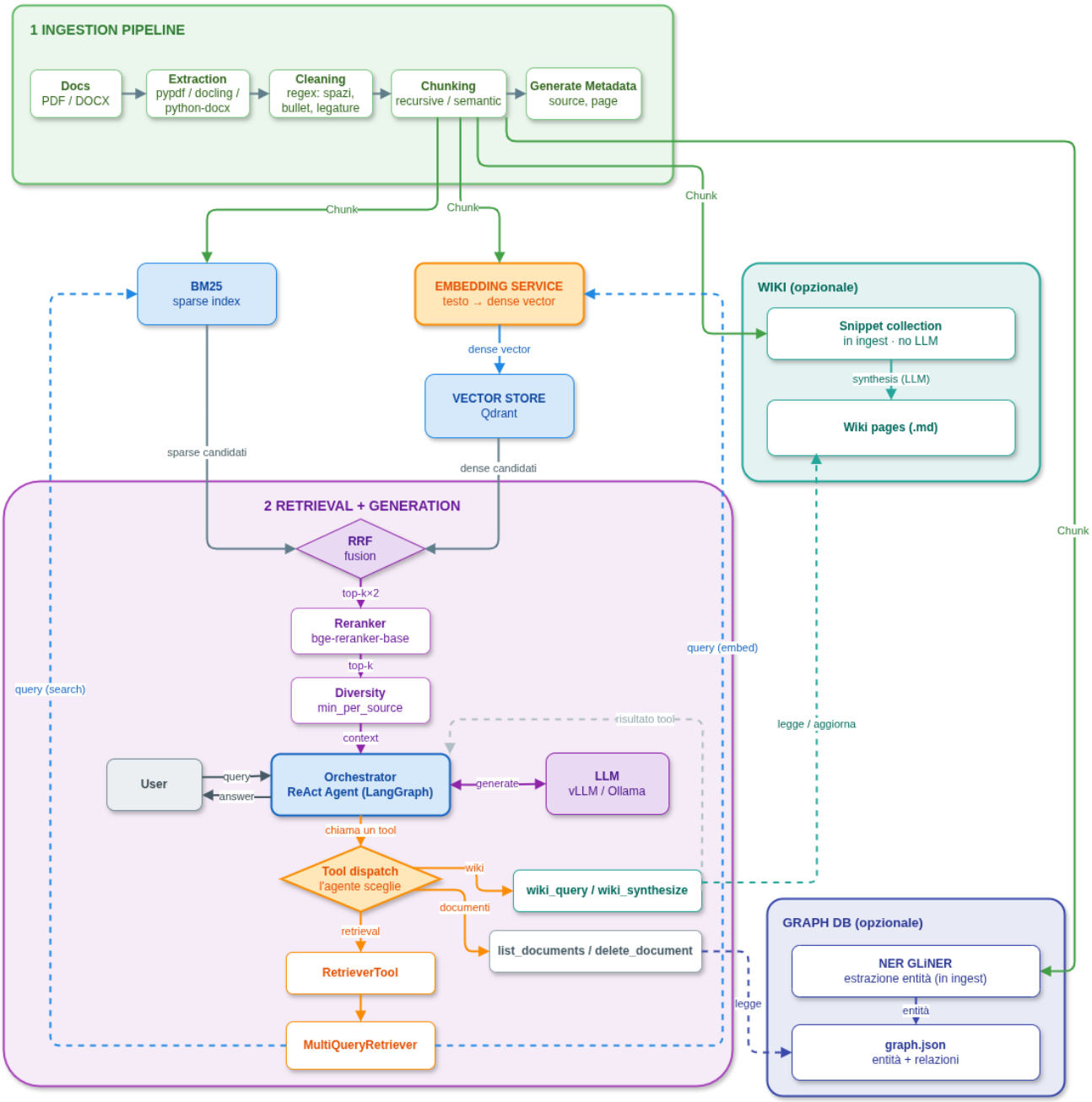


Figure 1: Detailed Vectoria pipeline

The diagram is supplied with the repository. The upload interface supports more file types than its PDF and DOCX labels show. The language model that writes answers is separate from the embedding model used for document search.

Indexing and answering workflows

Indexing reads the documents, divides their text into passages and creates searchable representations. Source metadata helps identify the files used in an answer. The embedding model and passage-splitting settings determine the index; changing them requires rebuilding it from the original files.

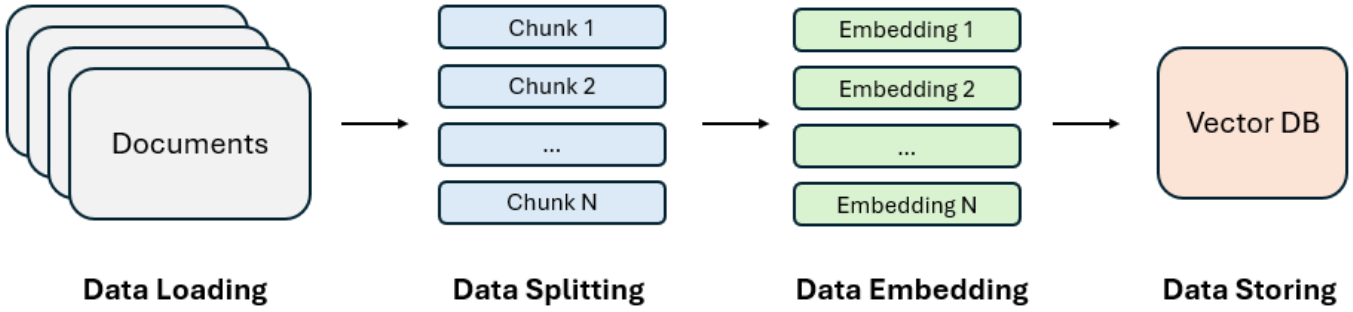


Figure 2: Build vector database workflow

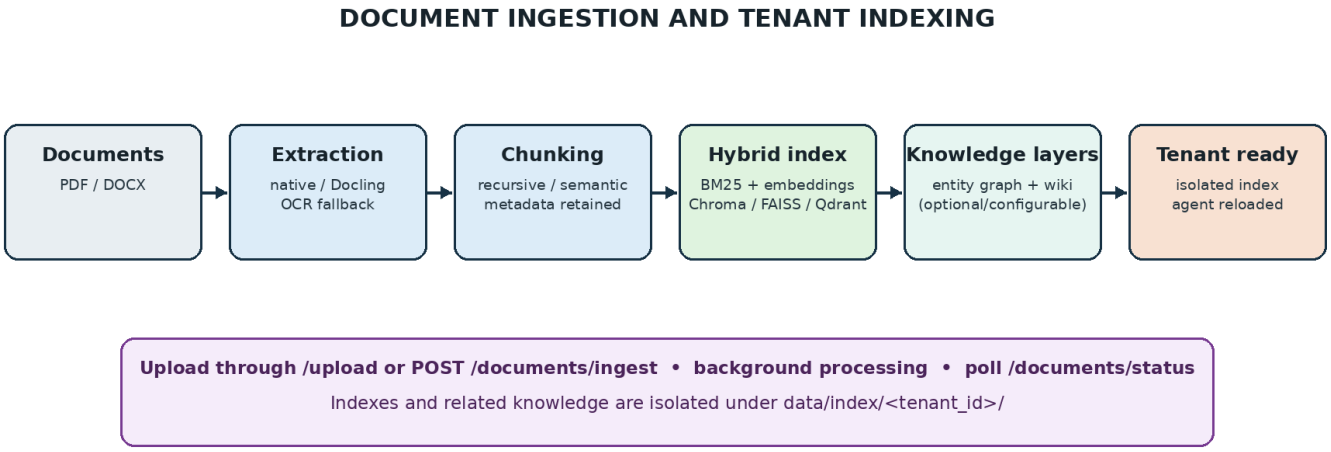


Figure 3: Document ingestion and tenant indexing workflow

At question time, Vectoria retrieves relevant passages and can rerank them before asking the language model to write an answer. The agent may also use document-management tools, the wiki or the entity graph when these are available.

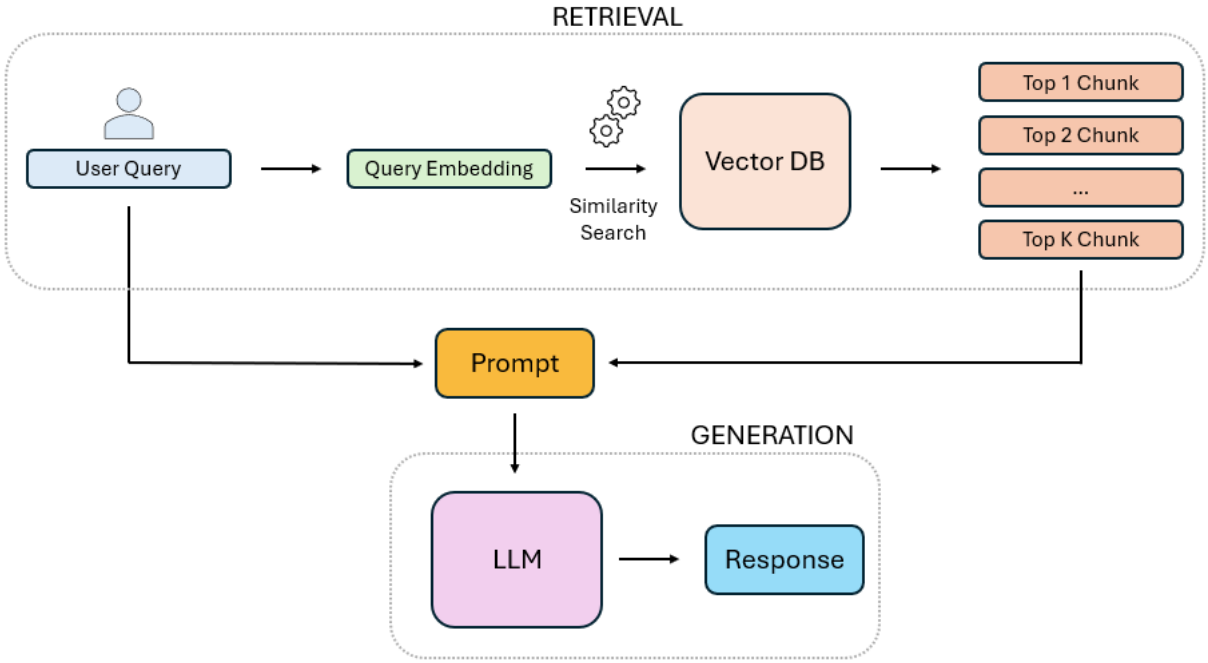


Figure 4: Inference workflow

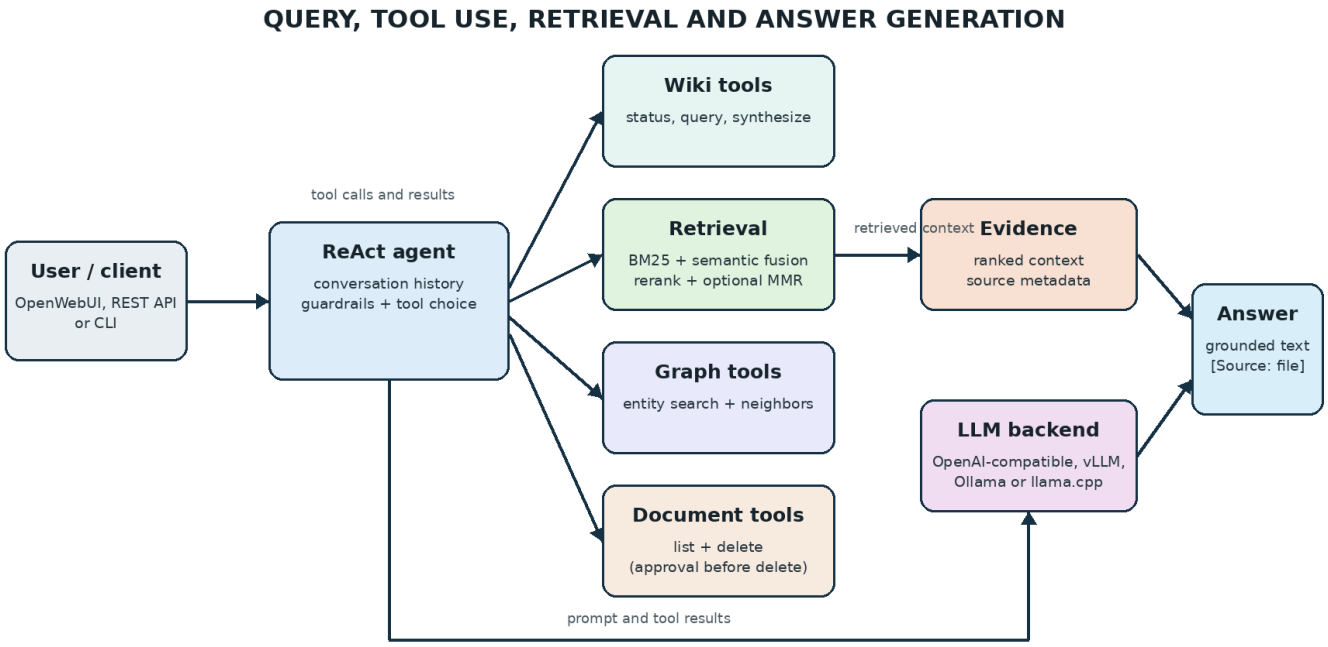


Figure 5: Query, tool use, retrieval and answer generation workflow

These two diagrams describe the basic search workflow. The full pipeline on the previous page also includes hybrid search, optional knowledge features and agent tools. Changing the answer-writing model alone does not require new document embeddings.

10. Appendix B: Advanced Configuration

Use the configuration source appropriate to the deployment:

Deployment	Configuration source
Direct host execution	etc/local/config.yaml, or a custom copy passed with --config
Docker	etc/docker/config.yaml, or a mounted custom configuration
Leonardo	ansible/roles/vectoria/vars/config.yml and its Jinja templates

For Leonardo, Ansible renders the application configuration into etc/hpc/default_config.yaml on the remote system. Editing a generated file is not a durable deployment change: redeployment can overwrite it.

The tables below describe supported settings. Defaults differ between local, Docker and HPC profiles; inspect the selected configuration file for its active values. The schema in vectoria_lib/core/config_schema.py defines accepted values, while the graph implementation accepts additional settings shown below.

Edit existing YAML sections rather than adding duplicate sections with the same name.

Environment variables and model providers

For an OpenAI-compatible endpoint, use:

```
LLM_PROVIDER=custom
LLM_BASE_URL=https://your-model-server.example/v1
```

```
LLM_API_KEY=replace-with-your-model-key
LLM_MODEL=replace-with-the-served-model-name
LLM_TIMEOUT=120
```

The model must support the tool calls used by the interactive agent.

Direct host execution also supports provider selection:

LLM_PROVIDER	Additional variables
custom	LLM_BASE_URL, LLM_API_KEY, LLM_MODEL
openai	OPENAI_API_KEY, LLM_MODEL
anthropic	ANTHROPIC_API_KEY, LLM_MODEL
google	GOOGLE_API_KEY, LLM_MODEL
ollama	OLLAMA_BASE_URL, LLM_MODEL

OLLAMA_BASE_URL is the server root, for example http://localhost:11434; provider selection appends /v1.

These provider-selection variables are read by the host application. The supplied Compose file does not forward LLM_PROVIDER or the provider-specific credentials into the container. For Docker, use the explicitly forwarded OpenAI-compatible URL/key/model variables, or modify Compose to forward the required variables before using another provider mode.

When using another provider on the host, start from etc/local/config.yaml. Its chat_history.summary_llm: null reuses the main model. A separate summary-model configuration must be updated independently if it names a different provider or endpoint.

The default Docker configuration contains chat_template_kwargs.enable_thinking. This extension is backend-specific. Removing it from the main model does not remove the extension added internally by wiki synthesis and query rewriting. Keep those features disabled when the endpoint rejects it.

General settings and model files

Setting	Purpose
vectoria_logs_dir	Writable log directory
log_level	DEBUG, INFO, WARNING, ERROR or CRITICAL
system_prompts_lang	eng or it
models_dir	Optional base directory for locally stored embedding, reranker and tokenizer models

When models_dir is set, Vectoria checks for model directories beneath it. If a matching directory is absent, the model name remains a Hugging Face model identifier.

Initial downloads may require network access and model authorization. Prepare caches before attempting an offline installation.

Extraction and chunking

Uploads are dispatched by file extension. The extraction backend controls how supported documents are processed.

Setting	Values or meaning
data_ingestion.extraction.backend	native, docling or auto
data_ingestion.multiprocessing	Enable preprocessing worker processes
data_ingestion.multiprocessing_workers	Worker count
data_ingestion.extraction.dump_doc_structure_on_file	Write extracted structure for inspection
data_ingestion.extraction.regexes_for_metadata_extraction	Configured metadata patterns
data_ingestion.regexes_for_replacement	Text-cleaning replacement rules
data_ingestion.chunking.method	recursive or semantic
data_ingestion.chunking.chunk_size	Recursive chunk size
data_ingestion.chunking.chunk_overlap	Recursive overlap
data_ingestion.chunking.tokenizer_name	Tokenizer used to measure recursive chunk size; null uses character counts
data_ingestion.chunking.separators	Ordered splitting separators
data_ingestion.chunking.is_separator_regex	Whether each separator is a regular expression
data_ingestion.chunking.dump_chunks_on_file	Write chunks for inspection

native does not perform OCR. auto starts with native PDF extraction and falls back to Docling when any page has insufficient extracted text. Therefore even an otherwise readable PDF may require Docling when one page has little text.

Semantic chunking additionally accepts:

Setting under data_ingestion.chunking	Purpose
semantic_embedding_model	Sentence-similarity model; defaults to the vector-store model
semantic_device	Device such as cpu or cuda
semantic_breakpoint_threshold_type	percentile, standard_deviation, interquartile or gradient
semantic_breakpoint_threshold_amount	Threshold value for the selected method
semantic_max_chunk_size	Optional maximum size; tokens when a tokenizer is configured, otherwise words

More preprocessing workers can load additional model copies and increase memory use. Begin with one worker when resources are limited.

Vector storage and embeddings

Setting under vector_store	Purpose

name	qdrant, chroma or faiss
model_name	Embedding model
device	Embedding device
normalize_embeddings	Normalize embedding vectors
embedding_batch_size	Embedding batch size
collection_name	Collection name for backends that use collections
qdrant_url	Remote Qdrant address; omit for local file-based Qdrant
qdrant_api_key	Remote Qdrant credential
qdrant_prefer_grpc	Prefer gRPC for remote Qdrant

Local Qdrant does not require a separately running Qdrant service.

Changing the embedding model requires rebuilding the index. Changes to chunking or extraction require reprocessing the original documents to apply consistently.

Remote Qdrant uses server-side collections rather than separate local database directories. Do not assume the local per-tenant directory layout provides isolation in remote mode. Configure and verify collection separation before using remote Qdrant for multiple tenants.

Retrieval and reranking

Setting under retriever	Purpose
enabled	Enable document retrieval
search_type	similarity, mmr or hybrid
top_k	Initial retrieval candidate count
fetch_k	Candidate pool for MMR search
lambda_mult	Relevance/diversity balance for MMR search
bm25_weight	Lexical contribution to hybrid retrieval
multi_query	Generate alternative search queries using the LLM
min_per_source	Minimum source-document representation; zero disables it
mmr_enabled	Enable additional diversity selection after reranking
mmr_lambda	Relevance/diversity balance for that additional selection
mmr_top_k	Final count for that additional selection; zero uses implementation defaults

Search-type MMR and post-reranking MMR are separate stages.

Reranker settings are:

```
reranker:
  enabled: true
  model_name: cross-encoder/mmarco-mMiniLMv2-L12-H384-v1
  device: cpu
  reranked_top_k: 2
```

Fetch enough candidates with retriever.top_k for the reranker and source-diversity settings to have useful alternatives.

full_paragraphs_retriever.enabled enables expansion to fuller paragraph context. Larger retrieved contexts can increase latency and model-context requirements.

Wiki

Wiki settings are under wiki:

Setting	Purpose
`enabled`	Enable the generated knowledge layer
`pending_threshold`	Pending-concept threshold for synthesis
`max_topics_per_doc`	Topic extraction limit per document
`max_topics_per_batch`	Topic limit per synthesis batch
`language_normalization`	Normalize extracted topic language
`similarity_threshold`	Topic similarity threshold
`canonical_topic_language`	it or en
`synthesis_context_chunks`	Context passages used during synthesis
`max_page_words`	Target maximum page length

Wiki processing uses the language model and embedding model and can continue after the document index becomes ready.

Enable wiki only after verifying the auxiliary request-option compatibility described above.

Entity graph

Install the graph extra before enabling graph extraction. The following example uses CPU extraction and spaCy relationship labels:

```
graph_store:
  enabled: true
  extraction_device: cpu
  model_path: urchade/gliner_multi-v2.1
  entity_types: null
  min_entity_confidence: 0.5
  min_count: 1
  min_pmi: 0.0
  relation_extraction: false
  relation_min_count: 2
  relation_batch_size: 15
  spacy_model: it_core_news_sm
  entity_resolution: false
  resolution_threshold: 0.92
```

entity_types: null asks the LLM to infer entity types. A fixed list avoids that inference step.

relation_extraction: false uses the spaCy labeling fallback where available. true requests relationship labels from the LLM and adds inference work.

The spaCy language model is installed separately from the Python graph extra. Its name must match spacy_model.

Graph generation is integrated into the upload/API indexing workflow. Building an index with the standalone CLI does not by itself populate the graph.

backend: networkx and relation_method: cooccurrence are accepted settings, but currently do not select alternative graph implementations.

Conversation history and guardrails

Setting under chat_history	Purpose
enabled	Enable conversation summarization middleware
summarize_after_n_messages	Summarization trigger
max_conv_messages	Messages retained by summarization
checkpoint_db_path	Persistent SQLite checkpoint location
summary_llm	Null to reuse the main model, a model-name string, or a separate inference configuration

Set checkpoint_db_path to null when persistent checkpoints are not wanted. This setting is separate from summarization. Approval workflows that pause and resume should retain a checkpointer.

max_tool_messages exists in the configuration schema but should not be relied on as an enforced retention control in the current agent implementation.

Guardrails are configured as:

```
agent_guardrails:
  tool_call_limit:
    enabled: true
    run_limit: 8
    thread_limit: null
    exit_behavior: continue
  human_approval:
    enabled: true
    tools: [delete_document]
```

exit_behavior accepts continue, error or end. thread_limit can impose a conversation-wide limit.

Inference settings and local model servers

Main inference settings include:

Setting under inference_engine	Purpose
name	Engine implementation
model_name	Served model name or local model identifier
base_url, api_key	Endpoint and credential
temperature, top_p	Sampling controls
max_tokens	Output budget
streaming	Streaming responses
request_timeout, max_retries	Request handling
model_kwargs	Backend-specific options; verify endpoint support

The recommended integration for vLLM, Ollama and llama-server is a tool-capable OpenAI-compatible endpoint with name: openai.

Vectoria can optionally start an existing llama-server executable. For host execution:

```
llama_server:
  enabled: true
  executable: /absolute/path/to/llama-server
  model: /absolute/path/to/model.gguf
  port: 3002
  ctx_size: 8192
  threads: 4
```

Configure its upstream connection in .env:

```
LLM_PROVIDER=custom
LLM_BASE_URL=http://localhost:3002/v1
LLM_API_KEY=none
LLM_MODEL=replace-with-the-served-model-name
```

The executable and model must already exist. Vectoria does not compile llama-server.

Additional settings include n_predict, n_gpu_layers, batch_size, ubatch_size, cache_type_k, cache_type_v, mlock and extra_args. Their suitability depends on the installed llama-server version and model.

In Docker, set MODELS_DIR and LLAMA_SERVER_DIR to the host directories mounted at /models and /llama-server; configure the executable and model using their container paths. The binary must be compatible with the container environment.

Direct in-process huggingface and llamacpp engines are implemented. The latter requires the llamacpp extra. These engines are not established substitutes for the tool-capable endpoint in the complete interactive agent workflow; validate their tool behavior before adopting them.

MLflow observability

Installing the observability extra does not enable tracking by itself.

For Docker, set in .env:

```
MLFLOW_TRACKING_URI=http://mlflow:5000
```

Start the profile and recreate services to apply environment changes:

```
docker compose --profile observability up -d --force-recreate
```

Open http://localhost:5000. MLflow stores data in its separate named volume.

For host execution, connect to a running tracking service using its host-reachable URL:

```
MLFLOW_TRACKING_URI=http://localhost:5000
```

Restart the API after changing the environment.

Applying configuration changes

Restart the host process after changing YAML or .env.

For Docker:

- A mounted YAML configuration requires a backend restart.
- Environment changes require container recreation.
- Configuration packaged inside the image requires rebuilding and recreating the backend.

Examples:

```
docker compose restart vectoria-api
docker compose up -d --force-recreate vectoria-api
docker compose up -d --build vectoria-api
```

For HPC, change the Ansible source settings and redeploy. Scheduler resources cannot be changed for an already running allocation.

Back up data before rebuilding indexes or changing storage backends. Preserve the original documents.

11. Appendix C: API and command line use

Authentication and collection selection

LLM_API_KEY authenticates Vectoria to the upstream language model.

PERMITTED_KEYS controls authentication to Vectoria itself. When nonempty, protected routes require an x-token header containing one permitted value. Multiple keys are comma-separated without spaces.

The upload page and terminal chat client do not send this header. OpenWebUI's standard Bearer key does not satisfy the check. Enabling PERMITTED_KEYS therefore requires an API client or a trusted gateway that supplies the required header.

The supplied OpenWebUI deployment disables its login screen. It does not provide a complete authenticated shared-service configuration.

Tenant IDs select document workspaces. They are not user accounts, and Vectoria does not bind permitted keys to individual tenants. Add an authorization layer when users must be restricted to particular collections.

On HPC, VECTORIA_API_TOKEN supplies the vLLM credential. It does not automatically enable Vectoria's PERMITTED_KEYS authentication.

Upload through the API

Set VECTORIA_TOKEN in your shell to a permitted Vectoria key. If authentication is disabled, omit the x-token headers below.

Upload a document:

```
curl -X POST http://localhost:8000/documents/ingest \
  -H "x-token: $VECTORIA_TOKEN" \
  -H "X-Tenant-ID: company-a" \
  -F "files=@document.pdf"
```

Check status:

```
curl http://localhost:8000/documents/status \
  -H "x-token: $VECTORIA_TOKEN" \
  -H "X-Tenant-ID: company-a"
```

Wait for status to become ready and agent_ready to become true.

List available models:

```
curl http://localhost:8000/v1/models \
  -H "x-token: $VECTORIA_TOKEN"
```

Send a question:

```
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "x-token: $VECTORIA_TOKEN" \
  -H "X-Tenant-ID: company-a" \
  -H "X-Thread-ID: company-a-example-1" \
  -d '{
    "model": "vectoria-company-a",
    "messages": [
      {
        "role": "user",
        "content": "Summarize document.pdf and identify the source."
      }
    ],
    "stream": false
  }'
```

Use a stable, distinct X-Thread-ID for each conversation. Start a new thread when changing collection or beginning an unrelated conversation.

The examples use local port 8000. For Leonardo's default tunnel, use port 8081.

Command-line server and terminal chat

Use the installation profiles in Section 6. For the minimal host profile:

```
uv run python bin/vectoria --config etc/local/config.yaml serve \
  --index-dir data/index \
  --upload-dir data/uploads \
  --host 127.0.0.1 \
  --port 8000
```

Upload through the browser or API before asking questions about documents.

With Vectoria token authentication disabled, start terminal chat:

```
uv run python bin/chat --tenant company-a
```

For a different API address:

```
uv run python bin/chat \
  --url http://localhost:8081 \
  --tenant company-a
```

Commands are:

- /aiuto: show help.
- /pulisci: start a fresh chat session.
- /esci: exit.

The CLI also provides build_index, generate and inference commands. Inspect their current options with:

```
uv run python bin/vectoria --config etc/local/config.yaml build_index --help
uv run python bin/vectoria --config etc/local/config.yaml generate --help
uv run python bin/vectoria --config etc/local/config.yaml inference --help
```

These standalone commands are distinct from API upload and tenant management. Do not assume a standalone index build also generates the API workflow's wiki or graph.

The older Streamlit GUI and start_build_index.sbatch workflow are not the deployment procedures described in this manual.

12. Application information and contributions

The application and this manual are released under the MIT open source license.


```
Copyright (c) 2024 EuroCC Italy, IT4LIA AI Factory, Cineca, Leonardo S.p.A.

Permission is hereby granted, free of charge, to any person
obtaining a copy of this software and associated documentation
files (the "Software"), to deal in the Software without
restriction, including without limitation the rights to use,
copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the
Software is furnished to do so, subject to the following
conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR
OTHER DEALINGS IN THE SOFTWARE.
```


This manual, *version 2.0*, refers to tag *v2.0* of repos:

https://github.com/IT4LIA-AIFactoryEU/vectoria



This project has been developed under the IT4LIA AI Factory funded project (Grant Agreement No. 101234224).

The first version of this application was developed by **EuroCC Italy** national competence center, with the specific contribution of **CINECA** and Leonardo S.p.A.

The project was realized thanks to:

Luca Babetto, Leonardo Baroncelli, Carolina Berucci, Mattia Buzzoni, Chiara Malizia, Eric Pascolo, Andrea Proia, Michele Visciarelli